

CampusMate

A Multilingual, Voice-Enabled College Query Assistant

Implementation Plan using Retrieval-Augmented Generation (RAG), Hybrid Search and LLMs

Document	Project Implementation Plan
Domain	Education / Student Services
Core Tech	RAG, Vector Embeddings, Hybrid Search, LLM
Version	1.0
Status	Proposal / Planning Stage

1. Introduction

Students and parents repeatedly ask colleges the same questions: admission dates, fee structure, hostel rules, exam timetables, placements, scholarships and more. The answers are scattered across PDFs, notices and websites, and administrative staff spend large amounts of time answering them manually.

CampusMate is an AI-powered assistant that answers such queries instantly by retrieving information from official college documents and generating grounded, cited responses. Unlike a basic FAQ bot, it supports regional languages, voice input, deadline reminders and a human-escalation path when it is not confident.

2. Objectives

- Provide accurate, 24/7 answers to college-related queries from official documents only.
- Reduce repetitive workload on the administrative office and help-desk.
- Support English, Hindi and Telugu so that more students and parents can use it comfortably.
- Show the source document for every answer to build trust and avoid hallucination.
- Give administrators a dashboard to upload documents and monitor unanswered questions.

3. Scope

In Scope	Out of Scope (this version)
Admissions, fees, courses, faculty, exam schedules, events, hostel, library, placements, scholarships	Personal student data (marks, attendance, fee dues) requiring login to the ERP
Web chat widget + voice input	Native mobile app (planned as future work)
Admin dashboard and analytics	Fine-tuning of a custom LLM
Multilingual Q&A (EN / HI / TE)	Payment processing or form submission

4. Feature Set

The table below compares the features planned for CampusMate. Items marked **New** are additions beyond a standard RAG college chatbot; items marked **Simplified** are intentionally reduced to keep the project achievable.

#	Feature	Description	Type
1	Document-grounded Q&A	Answers generated only from retrieved college documents using RAG.	Core
2	Hybrid search	Combines semantic (vector) search with keyword (BM25) search for exact terms such as course codes and dates.	New
3	Source citations	Each answer lists document name and page number.	New
4	Multilingual support	Detects language and replies in English, Hindi or Telugu.	New
5	Voice input / output	Browser speech-to-text for questions and optional text-to-speech for answers.	New
6	Conversation memory	Remembers the last few turns for follow-up questions (e.g. "What about the fees for that?").	Core
7	Confidence score and fallback	If retrieval confidence is low, bot says it is unsure and offers contact details.	New
8	Human escalation	Unanswered questions are logged and forwarded to the relevant department by email.	New
9	Deadline reminders	Students can subscribe to admission, exam-form and scholarship deadline alerts by email.	New
10	Quick-reply buttons	Suggested topics such as Admissions, Fees, Hostel, Placements.	Core
11	Admin dashboard	Upload/replace documents, re-index, view analytics and unanswered queries.	New
12	Feedback loop	Thumbs up/down on every answer to measure quality.	New
13	Model training	Using a hosted LLM through API instead of fine-tuning or training any model.	Simplified
14	Authentication	No student login in v1; only the admin panel is protected.	Simplified

5. System Architecture

CampusMate follows a standard RAG pipeline split into an offline **ingestion** stage and an online **query** stage.

5.1 Ingestion pipeline (offline)

Step	Component	Details
1	Document loader	Reads PDF, DOCX, HTML pages and CSV timetables uploaded by the admin or scraped from the college site.
2	Cleaning	Removes headers/footers, fixes encoding, runs OCR (Tesseract) on scanned notices.
3	Chunking	Splits text into ~500-token chunks with 80-token overlap; keeps metadata (title, page, category, date).
4	Embedding	Converts chunks to vectors with a multilingual embedding model.
5	Storage	Vectors in a vector database; raw text in a BM25 index; metadata in PostgreSQL.

5.2 Query pipeline (online)

Step	Component	Details
1	Input handler	Accepts typed text or speech-to-text output; detects language.
2	Query rewriting	Uses chat history to turn follow-ups into standalone queries.
3	Hybrid retrieval	Top-k from vector search and BM25, merged using Reciprocal Rank Fusion.
4	Re-ranking	Cross-encoder re-ranks candidates; best 4-5 chunks are kept.
5	Generation	LLM receives chunks plus strict instructions to answer only from context and cite sources.
6	Guardrails	Checks confidence threshold, filters off-topic or unsafe prompts.
7	Response	Returns answer, citations, suggested follow-ups and a feedback widget.

5.3 Flow summary

User question -> Language detection -> Query rewriting -> Hybrid retrieval (Vector + BM25) -> Re-ranker -> LLM with context -> Guardrails / confidence check -> Answer with citations (or escalation to staff)

6. Technology Stack

Layer	Technology	Reason
Frontend	React + Tailwind CSS, Web Speech API	Responsive chat widget with voice support
Backend	Python, FastAPI	Async APIs, strong AI ecosystem
Orchestration	LangChain or LlamaIndex	Ready-made RAG components
Embeddings	multilingual-e5 / BGE-M3 (open source)	Works well for Hindi and Telugu
Vector DB	ChromaDB (dev) / Qdrant (production)	Simple setup, metadata filtering
Keyword search	rank-bm25 / OpenSearch	Exact match for codes, names and dates
LLM	Claude, GPT or open model via API	High quality grounded answers
Database	PostgreSQL	Chat logs, feedback, subscriptions, admin users
Scheduler / Email	APScheduler + SMTP	Deadline reminders and escalations
Deployment	Docker, Nginx, cloud VM	Easy and reproducible hosting

7. Data Sources and Knowledge Base

- Prospectus, brochures and admission notifications
- Fee structure and scholarship circulars
- Academic calendar, exam timetables and syllabus copies
- Hostel, library, transport and anti-ragging policies
- Placement statistics and recruiter lists
- Faculty and department directory, contact numbers
- Existing FAQ sheets maintained by the office

Each document is tagged with category and validity date so outdated circulars can be down-ranked or removed.

8. Database Design (PostgreSQL)

Table	Key fields
documents	id, title, category, file_path, uploaded_by, valid_from, valid_till, status
chunks	id, document_id, page_no, text, vector_id
chat_sessions	id, language, started_at, channel
messages	id, session_id, role, content, sources, confidence, created_at
feedback	id, message_id, rating, comment
escalations	id, question, department, status, assigned_to, resolved_at
subscriptions	id, email, topic, language, created_at
admins	id, name, email, password_hash, role

9. API Endpoints

Method	Endpoint	Purpose
POST	/api/chat	Send a question, receive answer + citations
POST	/api/feedback	Submit thumbs up/down
POST	/api/subscribe	Subscribe to deadline alerts
POST	/api/admin/documents	Upload a document and trigger indexing
DELETE	/api/admin/documents/{id}	Remove a document and its vectors
GET	/api/admin/analytics	Usage, top topics, unanswered questions
GET	/api/admin/escalations	List pending escalations

10. Prompt Design

The system prompt constrains the model to the retrieved context:

You are CampusMate, the official assistant of the college.
Answer ONLY using the CONTEXT below. If the answer is not in the context, say you are not sure and share the office contact.
Reply in the same language as the user. Keep answers short and clear.
End with the sources used, formatted as [Document, Page].

11. Implementation Roadmap

Phase	Duration	Activities	Deliverable
1. Planning	Week 1	Requirements, collect documents, finalize stack	SRS and document set
2. Data pipeline	Weeks 2-3	Loaders, OCR, chunking, embeddings, vector store	Searchable knowledge base
3. RAG core	Weeks 4-5	Hybrid retrieval, re-ranker, prompts, citations	Working CLI / API prototype
4. Chat UI	Weeks 6-7	React widget, quick replies, voice input, multilingual	Usable web chatbot
5. Admin and extras	Weeks 8-9	Dashboard, feedback, escalation, reminders	Admin panel and alerts
6. Testing	Week 10	Accuracy evaluation, load and security testing	Test report
7. Deployment	Week 11	Docker, hosting, pilot with students	Live pilot
8. Review	Week 12	Collect feedback, fix issues, documentation	Final report

12. Testing and Evaluation

Metric	Method	Target
Answer accuracy	100 curated questions checked by staff	85% or higher
Retrieval hit-rate@5	Check if correct chunk is in top 5	90% or higher
Groundedness	Manual check that answers match sources	No fabricated facts
Response time	Average end-to-end latency	Under 4 seconds
Language quality	Native speakers rate Hindi/Telugu replies	4 / 5 or higher
User satisfaction	Thumbs-up ratio from pilot	75% or higher

13. Risks and Mitigation

Risk	Impact	Mitigation
Hallucinated answers	High	Strict context-only prompt, confidence threshold, citations
Outdated documents	High	Validity dates, admin re-index workflow, expiry alerts
Poor OCR on scanned notices	Medium	Better OCR settings, manual review of key documents
Weak regional-language quality	Medium	Multilingual embeddings, native-speaker testing
Prompt injection / misuse	Medium	Input filtering, system prompt isolation, rate limiting
API cost or downtime	Medium	Caching frequent queries, fallback to open-source model
Privacy of chat logs	Medium	No personal data collection, anonymised logs, retention limit

14. Security and Privacy

- HTTPS everywhere; admin panel behind login with hashed passwords and role-based access.
- Rate limiting and CAPTCHA on public endpoints.
- Chat logs anonymised; email addresses stored only for subscribers and deletable on request.
- Regular backups of the database and uploaded documents.

15. Future Enhancements

- Integration with the college ERP for personalised answers (attendance, results, fee dues) after login.
- WhatsApp and Telegram channels.
- Campus map and navigation assistant.
- Automatic detection of new notices on the college website.
- More Indian languages and offline-capable kiosk mode.

16. Conclusion

CampusMate extends the idea of a document-based college chatbot with hybrid retrieval, citations, multilingual and voice access, reminders and a human fallback, while avoiding unnecessary complexity such as model training and student logins. Built in roughly twelve weeks with open-source tools and a hosted LLM, it can reduce office workload and give students faster, more reliable answers.

References

- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", 2020.
- LangChain and LlamaIndex documentation.
- ChromaDB and Qdrant documentation.
- FastAPI documentation.